COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	September 16, 2026	**DATE SUBMITTED:**	September 16, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Engr. Jimlord Quejado

Activity 8.1 **Sorting Algorithms Pt. 2**

1. PROCEDURE OUTPUT

Table 8-1:

Code + Console Screenshot	Code:
	`int main() {`
	`  int originalDataset[SIZE];`
	`  int testArray[SIZE];`
	`  srand(time(0));`
	`  std::cout << "Original Unsorted Dataset: \n";`
	`  for (int i = 0; i < SIZE; i++) {`
	`  originalDataset[i] = rand() % 100;`
	`  }`
	`  printArray(originalDataset, SIZE);`
	
	Output:
	`Original Unsorted Dataset:`
	`5 99 53 26 47 34 80 25 32 43 86 88 72 90 82 19 10 75 50 35 84 57 55 4 40 29 7 58 63 67 5 79 53 80 24 69 21 82 36 59 22 12 45 30 76 77 4 74 0`
	`6 7 79 60 53 64 9 38 73 61 46 79 72 46 34 46 36 51 35 89 21 88 10 20 45 63 69 10 53 9 35 8 60 55 65 18 5 62 11 71 22 5 30 36 58 38 83 46 22`
	`59 4`

Observations	The dataset successfully initializes 100 unsorted integers ranging between 0 and 99 using dynamic random value generation (rand() % 100. The array displays an arbitrary order, serving as the identical baseline dataset across all sorting tests.

Table 8-2:

Code + Console Screenshot	Code:

```cpp
template <typename T>
void shellSort(T arr[], int size) {
    for (int interval = size / 2; interval > 0; interval /= 2) {
        for (int i = interval; i < size; i++) {
            T temp = arr[i];
            int j;
            for (j = i; j >= interval && arr[j - interval] > temp; j -= interval) {
                arr[j] = arr[j - interval];
            }
            arr[j] = temp;
        }
    }
}
```

```cpp

    copyArray(originalDataset, testArray, SIZE);
    shellSort(testArray, SIZE);
    std::cout << "Shell Sort Result:\n";
    printArray(testArray, SIZE);

```

Output:

```
Shell Sort Result:
0 4 4 4 5 5 5 5 6 7 7 8 9 9 10 10 10 11 12 18 19 20 21 21 22 22 22 24 25 26 29 30 30 32 34 34 35 35 35 3
6 36 36 38 38 40 43 45 45 46 46 46 46 47 50 51 53 53 53 53 55 55 57 58 58 59 59 60 60 61 62 63 63 64 65
67 69 69 71 72 72 73 74 75 76 77 79 79 79 80 80 82 82 83 84 86 88 88 89 90 99
```

Observations	Shell Sort reduces the 100 unsorted elements into ascending order by comparing elements separated by decreasing interval gaps (starting at SIZE / 2 down to 1). It rearranges distant elements efficiently early on, requiring minimal adjustments during its final insertion-sort phase.

Table 8-3

Code + Console Screenshot	Code:

```
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
```

```
43      while (i < n1) {
44          arr[k] = L[i];
45          i++;
46          k++;
47      }
48
49      while (j < n2) {
50          arr[k] = R[j];
51          j++;
52          k++;
53      }
54
55      delete[] L;
56      delete[] R;
57  }
```

```
59  template <typename T>
60  void mergeSort(T arr[], int left, int right) {
61      if (left < right) {
62          int middle = left + (right - left) / 2;
63          mergeSort(arr, left, middle);
64          mergeSort(arr, middle + 1, right);
65          merge(arr, left, middle, right);
66      }
67  }
```

```
37      copyArray(originalDataset, testArray, SIZE);
38      mergeSort(testArray, 0, SIZE - 1);
39      std::cout << "Merge Sort Result: \n";
40      printArray(testArray, SIZE);
41
```

Output:

```
Merge Sort Result:
0 4 4 4 5 5 5 5 6 7 7 8 9 9 10 10 10 11 12 18 19 20 21 21 22 22 22 24 25 26 29 30 30 32 34 34 35 35 35 3
6 36 36 38 38 40 43 45 45 46 46 46 46 47 50 51 53 53 53 53 55 55 57 58 58 59 59 60 60 61 62 63 63 64 65
67 69 69 71 72 72 73 74 75 76 77 79 79 79 80 80 82 82 83 84 86 88 88 89 90 99
```

Observations	Merge Sort recursively divides the 100-element array into single-element sub-arrays (O(log n) splits) and combines them back using a temporary array dynamically in sorted order (O(n) merge phase). The dataset is sorted reliably with a predictable O(n log n) performance.

Table8-4:

Code + Console Screenshot	Code:

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

```cpp
    copyArray(originalDataset, testArray, SIZE);
    quickSort(testArray, 0, SIZE - 1);
    std::cout << "Quick Sort Result: \n";
    printArray(testArray, SIZE);
```

Output:

Observations	Quick Sort selects a pivot element to partition the 100-element array in-place, placing smaller values to the left and larger values to the right. By recursively applying this partition scheme across left and right sub-arrays, the entire sequence achieves complete ascending order without requiring extra memory allocation.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:
Question: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, after the partition() function places the pivot element into its correct sorted position, the resulting left and right sub lists can be sorted using alternative sorting algorithms such as Shell Sort or Insertion Sort instead of standard recursive Quick Sort Calls

Code:

```
92
93  template <typename T>
94  void hybridQuickSort(T arr[], int low, int high){
95      if(low < high) {
96          int pivotIndex = partition(arr, low, high);
97
98          int leftSize = pivotIndex - low;
99          int rightSize = high - pivotIndex;
100
101         if(leftSize > 1){
102             shellSort(&arr[low], leftSize);
103         }
104         if(rightSize > 1){
105             shellSort(&arr[pivotIndex + 1], rightSize);
106         }
107     }
108 }
109
```

```
46
47      std::cout << "Problem 1: Hybrid Quick Sort\n";
48
49      int prob1Array[] = {48, 12, 87, 34, 4, 53, 29};
50      int prob1Size = 7;
51
52      std::cout << "Before Hybrid Quick Sort: ";
53      printArray(prob1Array, prob1Size);
54
55      hybridQuickSort(prob1Array, 0, prob1Size - 1);
56
57      std::cout <<"After Hybrid Quick Sort: ";
58      printArray(prob1Array, prob1Size);
59
```

Problem 2
Question: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

For the given dataset, Quick Sort provides the fastest real-world time performance, while both Quick Sort and Merge Sort achieve an O(NlogN) overall time complexity because their divide-and-conquer strategy recursively splits an array of N elements into logN sub-tree levels, processing all N elements across each level during partitioning or merging steps.

Conclusion:
In this laboratory activity, I successfully implemented and evaluated three fundamental sorting algorithms—Shell Sort, Merge Sort, and Quick Sort—using C++ templates on a dataset of 100 random integers. Through my implementation and testing, I observed that Shell Sort improves upon basic quadratic sorts by using decreasing interval gaps to minimize element movements early in the sorting process. I verified that Merge Sort reliably guarantees O(N log N) time performance across all cases by recursively dividing the dataset and merging sub-arrays, though it requires auxiliary memory to do so. Quick Sort proved to be the most efficient in-place algorithm for unsorted data, leveraging dynamic partitioning to achieve O(N log N) average performance. Additionally, my supplementary implementation demonstrated that Quick Sort can be effectively hybridized with other sorting techniques, such as Shell Sort, to process sub-lists after partitioning. Overall, this experiment broadened my understanding of algorithmic complexity, memory usage, and the practical trade-offs involved in selecting sorting algorithms for different dataset conditions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code debugging assistance, explanation of concepts, formatting and grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was utilized to aid in troubleshooting syntax issues, verifying boundary conditions in the searching and sorting header files, and polishing the written observations and conclusion for proper formatting and clarity. The tool functioned strictly as an advisory and supporting guide; it did not substitute for original problem-solving, custom code development, data analysis, or the independent execution of the lab requirements.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.